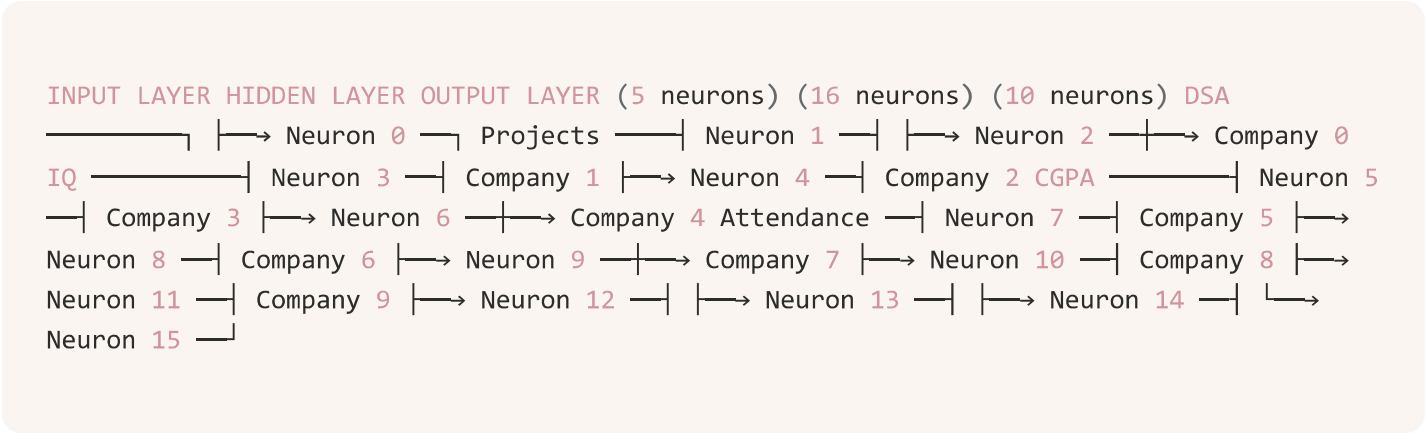
Multi class classification

Our neural network



forward pass

```
Student: [85, 5, 130, 9.0, 95] │ ┌──────┴──────┐ │ W1 (16 × 5) │ ← 80 weights + 16
biases │ + ReLU │ └──────┬──────┘ │ a1: [0, 2.3, 0, 1.7, 0.5, 0, 3.1, 0, 1.2, ...] (16
numbers) │ ┌──────┴──────┐ │ W2 (10 × 16) │ ← 160 weights + 10 biases └──────┬──────┘
│ z2: [-2.1, -1.5, 0.3, 1.2, 0.8, -0.4, 2.1, 5.3, 1.1, -0.9] │ ┌──────┴──────┐ │
Softmax │ └──────┬──────┘ │ a2: [0.1%, 0.1%, 0.6%, 1.5%, 1.0%, 0.3%, 3.7%, 91%, 1.4%,
0.2%] ↑ Prediction: Company 7
```

The complete backward flow

```
Error │ ▼ dz2 = predicted - actual ← Step 1: output error │ ┌──────┴──────┐ ▼ ▼
dW2 = dz2 × a1 db2 = dz2 ← Step 2: blame W2 │ ▼ da1 = W2ᵀ × dz2 ← Step 3: push blame to
hidden │ ▼ dz1 = da1 × ReLU'(z1) ← Step 4: blame through ReLU (pass if z1>0, kill if
z1≤0) │ ┌──────┴──────┐ ▼ ▼ dW1 = dz1 × x db1 = dz1 ← Step 5: blame W1
```

Case 1: Train 90%, Test 60% (big gap = overfitting)

The model memorized the training data but can't generalize. It learned noise instead of patterns — like a student who memorized 100 specific problems but can't solve a new one.

What to do:

Get more data. More examples = harder to memorize, forces the model to learn actual patterns instead of specific cases.

Reduce model size. Your model has too much capacity for the amount of data. Fewer hidden neurons means fewer weights, which means the model is forced to find simple, general patterns instead of memorizing every training sample.

Add regularization (dropout). During training, randomly "kill" some neurons (set their output to 0) each batch. This forces the network to not rely on any single neuron too heavily — it has to spread the learning across multiple neurons. Like studying with random pages torn out of your textbook — you can't memorize, you have to understand.

Early stopping. Track test accuracy every epoch. Stop training when test accuracy starts dropping, even if train accuracy is still going up. The moment test drops = the moment memorization begins.

Case 2: Train 62%, Test 60% (small gap = underfitting)

The model can't even learn the training data properly. It's not smart enough for the problem.

What to do:

Add more hidden neurons. Go from 16 → 32 → 64. Give the model more "questions" it can ask about each student.

Add another hidden layer. Let the model learn "patterns of patterns" — more complex relationships.

Train longer. Maybe 500 epochs isn't enough. Check if loss is still decreasing at epoch 500 — if yes, keep going.

Lower the learning rate. Maybe the model is taking steps that are too big and jumping over the optimal point. Smaller steps = more precise convergence.

Check your features. Maybe the 5 features you have aren't informative enough. Add better features.

Case 3: Train 60%, Test 60%, Ceiling 65%

Both are low, but close to the data ceiling. The model is doing its best — the data is the limit.

What to do:

Add more features. The current features can't separate the classes. You need more information — interview score, internship experience, communication skills.

Reduce the number of classes. Maybe 10 companies is too granular. Group similar companies: Company 0-3 → "Tier C", Company 4-6 → "Tier B", Company 7-9 → "Tier A". Now it's a 3-class problem instead of 10 — much easier to separate.

The decision tree

```
Test accuracy is low │ ▼ Check: Train accuracy vs Test accuracy │ ├── Train >> Test (90% vs 60%) │ OVERFITTING │ → More data │ → Smaller model │ → Dropout │ → Early stopping │ ├── Train ≈ Test (62% vs 60%) │ UNDERFITTING │ │ │ ├── Check: Is ceiling close? (ceiling ~65%) │ │ DATA IS THE LIMIT │ │ → Add features │ │ → Reduce classes │ │ │ └── Ceiling is high (ceiling ~90%) │ MODEL IS THE LIMIT │ → More neurons │ → More layers │ → Train longer │ → Lower learning rate │ └── Train < Test SOMETHING IS BROKEN → Check for bugs in code → Check data loading → Check normalization
```

This decision tree is what every ML engineer runs in their head when accuracy is bad. Worth showing on your Coder Army stream — most tutorials teach you how to build models, very few teach you **how to debug them**.